Database Systems – ISYS2014

Vietnam Tourism Database - Report

Henry Phan

23028920

October 2026

Introduction

In this assignment, I have developed a database system that uses data from Vietnam's National Statistics Office, centred on tourism, specifically its economic aspects. I first developed ER diagrams, database schemas, and tables for the database system. I then implemented these schemas on a MySQL server using SQL scripts that created and populated the tables. I have also written a Python program that connects to the database, allowing users to run predefined queries with their own inputs.

Design of Database

Entity	Primary key/s	Attribute/s
Year	year	
Province	provinceId	name
Region	regionId	name
Country	countryId	name continent
TransportMode	transportId	mode
ExpenditureCategory	categoryId	category
TouristType	typeId	type

Relationship sets	Between which entities	Attribute/s
ArrivalByNationality	Year Country	value
ArrivalByTransport	Year TransportMode	value
ProvincialTurnover	Year Province	value
ExpenditureByCategory	Year ExpenditureCategory TouristType	value, currency
Contains	Region Province	
Audit_ArrivalByNationality	ArrivalByNationality	old_arrival, new_arrival, audit_time

Relationship sets	Cardinality constraints
ArrivalByNationality	One year can have arrivals from many nationalities

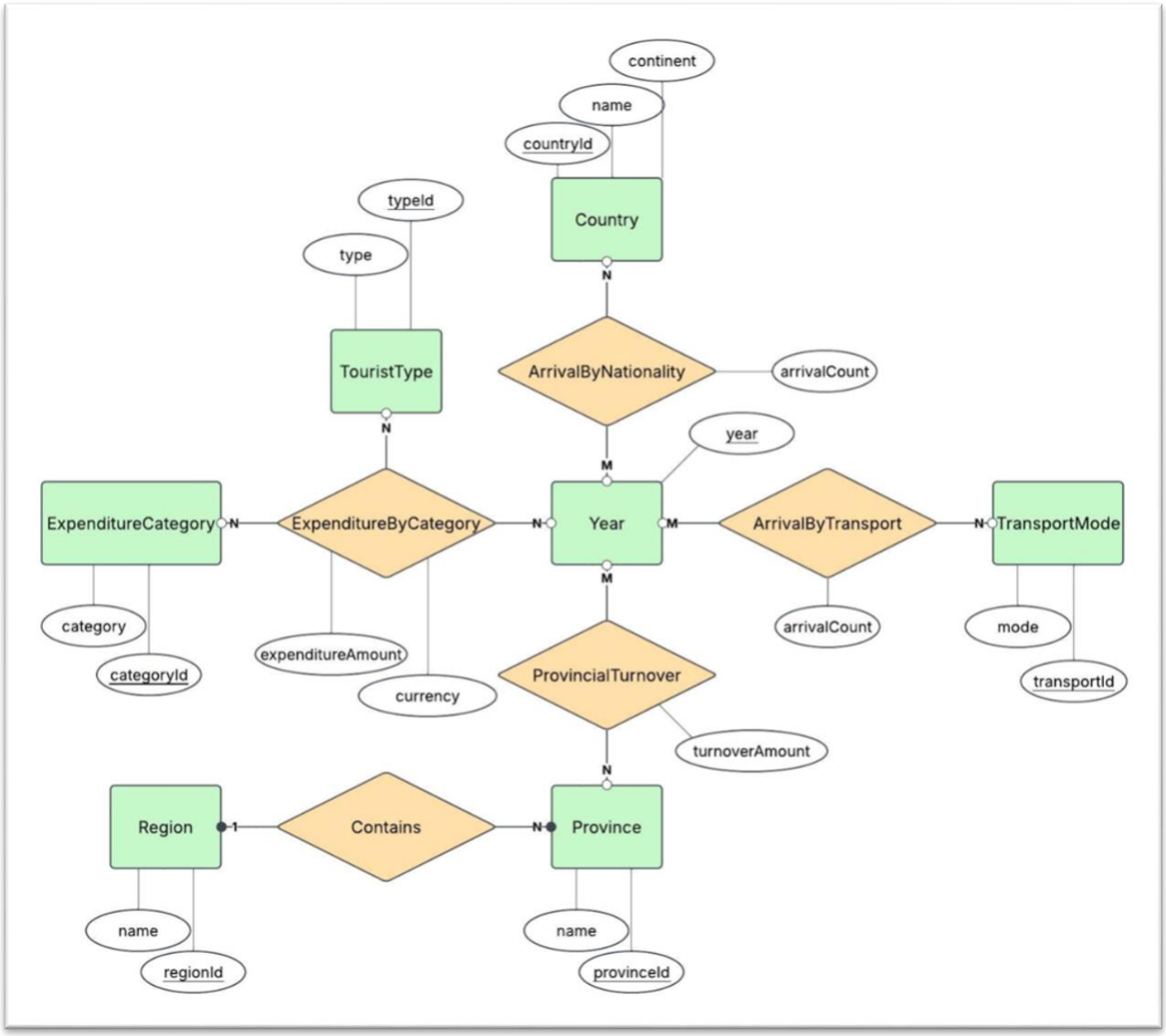
	One nationality can arrive for multiple years (M: N)
ArrivalByTransport	One year can have arrivals through many transport types One transport type can be used over many years (M: N)
ProvincialTurnover	One year can record turnovers for many provinces One province can have its turnover recorded over many years (M: N)
ExpenditureByCategory	A year-specific expenditure category can be recorded for multiple tourist types A year-specific tourist type can be recorded for multiple expenditure categories A tourist type with a specific expenditure category can be recorded over many years. (N:N:N)
Contains	A region can contain multiple provinces. A province belongs to only one region. (1: N)
Audit_ArrivalByNationality	A audit can only refer to one ArrivalByNationality record. An ArrivalByNationality record can be updated and auditted many times. (1:N)

Relationship sets	Participation constraints
ArrivalByNationality	A year instance does not have to be associated with a nationality. A nationality does not have to be associated with a year. (Partial: Partial)
ArrivalByTransport	A year instance does not have to be associated with a transport mode. A transport mode does not have to be associated with a year. (Partial: Partial)
ProvincialTurnover	A year instance does not have to be associated with a province.

	A province instance does not have to be associated with a year. (Partial: Partial)
ExpenditureByCategory	A year may exist without any expenditure observation. An expenditure category may exist without any observation. A tourist type may exist without expenditure observation. (Partial participation by all)
Contains	A region must contain at least one province. A province must be contained by a region. (Total: Total)
Audit_ArrivalByNationality	An audit must refer to an ArrivalByNationality record. An ArrivalByNationality record can exist without having an audit. (Total: Partial)

Assumptions: The database may contain years, transport types, nationalities, provinces, expenditure categories, and tourist types without corresponding observations. A province always belongs to a region, and a region must contain at least one province.

ER Diagram:



Mapping to schema notes:

- The ‘Contains’ relationship will be reduced so that ‘Province’ will refer to regionId from ‘Region’.
- The other many-to-many relationships are all kept as separate relationship entities, using the primary keys as part of their composite keys.
- Audit_ArrivalByNationality is not in the ER model, as it is an implementation-stage extension and not part of the original design.

Relational schema:

- Year(year)
- Region(regionId, name)
- Province(provinceId, name, regionId)
- Country(countryId, name, continent)
- TransportMode(transportId, mode)
- ExpenditureCategory(categoryId, category)
- TouristType(typeId, type)

- ArrivalByNationality(year, countryId, arrivalCount)
    - FK year REF Year(year),
    - FK countryId REF Country(countryId)
- ArrivalByTransport(year, transportId, arrivalCount)
    - FK year REF Year(year),
    - FK transportId REF TransportMode(transportId)
- ProvincialTurnover(year, provinceId, turnoverAmount)
    - FK year REF Year(year),
    - FK provinceId REF Province(provinceId)
- ExpenditureByCategory(year, categoryId, typeId, expenditureAmount, currency)
    - FK year REF Year(year),
    - FK categoryId REF ExpenditureCategory(categoryId),
    - FK typeId REF TouristType(typeId)
- Audit_ArrivalByNationality(auditId, year, countryId, old_arrival_count, new_arrival_count, audit_time)
    - FK (year, countryId) REF ArrivalByNationality(year, countryId)

Data description:

Year			
Attribute name	Data type	Description	Constraints
year	INT	The year number	PK, NOT NULL

Province			
Attribute name	Data type	Description	Constraints
provinceId	INT	ID no. for province	PK, NOT NULL AUTO_INCREMENT
name	VARCHAR(50)	Name of province	NOT NULL
regionId	INT	ID no. for region	FK, NOT NULL

Region			
Attribute name	Data type	Description	Constraints
regionId	INT	ID no. for region	PK, NOT NULL AUTO_INCREMENT
name	VARCHAR(50)	Name of region	NOT NULL

Country			
Attribute name	Data type	Description	Constraints
countryId	INT	ID no. for country	PK, NOT NULL AUTO_INCREMENT
name	VARCHAR(50)	Name of country	NOT NULL
continent	VARCHAR(50)	Country's continent	

TransportMode			
Attribute name	Data type	Description	Constraints
transportId	INT	ID no. for transport	PK, NOT NULL AUTO_INCREMENT
mode	VARCHAR(50)	Name of mode	NOT NULL

ExpenditureCategory			
Attribute name	Data type	Description	Constraints
categoryId	INT	ID no. for category	PK, NOT NULL
category	VARCHAR(50)	Category name	NOT NULL

TouristType			
Attribute name	Data type	Description	Constraints
typeId	INT	Tourist type ID	PK, NOT NULL AUTO_INCREMENT
type	VARCHAR(50)	Type of tourist	NOT NULL

ArrivalByNationality			
Attribute name	Data type	Description	Constraints
year	INT	Recorded arrival year	PK(composite), NOT NULL
countryId	INT	Country associated with recorded arrival	PK(composite), NOT NULL

arrivalCount	INT	Count of arrivals	NOT NULL arrivalCount >= 0

ArrivalByTransport			
Attribute name	Data type	Description	Constraints
year	INT	Recorded arrival year	PK(composite), NOT NULL
transportId	INT(1)	Transport mode of the recorded arrival	PK(composite), NOT NULL
arrivalCount	INT	Count of arrivals	NOT NULL arrivalCount >= 0

ProvincialTurnover			
Attribute name	Data type	Description	Constraints
year	INT	Recorded arrival year	PK(composite), NOT NULL
provinceId	INT	Province associated with the turnover	PK(composite), NOT NULL
turnoverAmount	DECIMAL(15,2)	Turnover in Bill. VND	NOT NULL amount >= 0

ExpenditureByCategory			
Attribute name	Data type	Description	Constraints
year	INT	Recorded year of the expenditure	PK(composite), NOT NULL
categoryId	INT	Category of the expenditure	PK(composite), NOT NULL
typeId	INT	The tourist type of the expenditure	PK(composite), NOT NULL
expenditureAmount	DECIMAL(15,2)	Expenditure amount	NOT NULL, amount >= 0
currency	CHAR(3)	Currency type	NOT NULL

Audit_ArrivalByNationality			
Attribute name	Data type	Description	Constraints
auditId	INT	ID no. for the audit	PK, NOT NULL AUTO_INCREMENT
year	INT	PK of updated rec.	FK, NOT NULL
countryId	INT	PK of updated rec.	FK, NOT NULL
old_arrival	INT	Previous arrival	

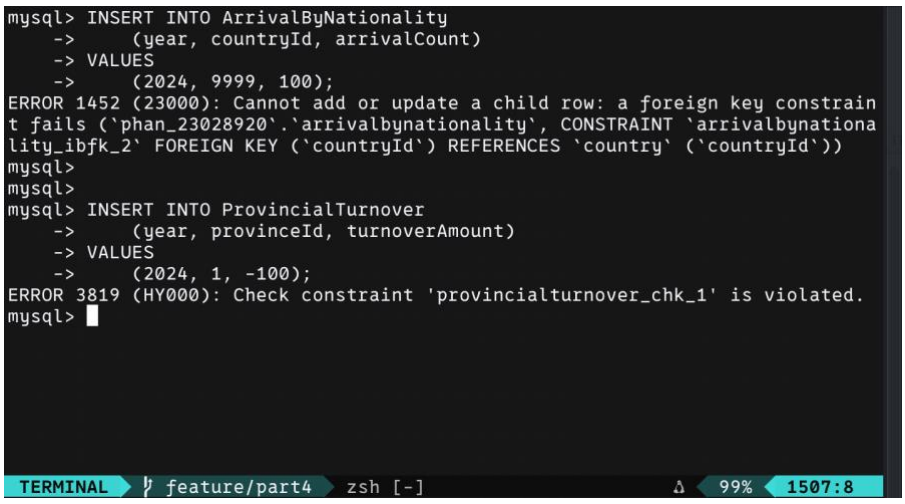
new_arrival	INT	Updated-to arrival	
audit_time	TIMESTAMP	Audit timestamp	NOT NULL

Database implementation:

Part2_1.sql: This SQL script implements all of the tables above into the database. Attributes are given the proper constraints as stated above. Below is a screenshot of all the tables after running this script.



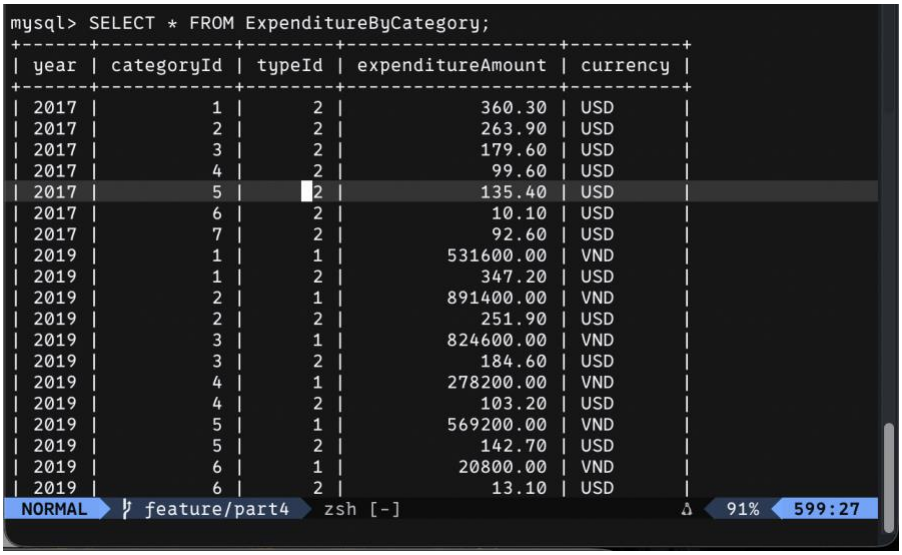
Tables have also been given appropriate constraints, such as ensuring turnover amounts do not go below 0, as well as referential integrity constraints on relationship entities. The screenshot below shows how these constraints are enforced:



Part2_2.sql: This script populates the 11 tables with data through both INSERT INTO and LOAD INFILE statements, with the audit table being empty initially. All data are retrieved from Vietnam's National Statistics website. The specific tables used were:

- “Number of foreigner arrivals to Viet Nam by some nationalities”
- “Average expenditure per domestic tourist by item”
- “Average expenditure per international tourist to Viet Nam”
- “Number of foreigner arrivals to Viet Nam”
- “Turnover of travelling at current prices by province”

The data was transformed and restructured to be used by both INSERT INTO and LOAD INFILE statements. Below is the content of ExpenditureByCategory after the data is loaded:



```
mysql> SELECT * FROM ExpenditureByCategory;
+------+------------+--------+-------------------+----------+
| year | categoryId | typeId | expenditureAmount | currency |
+------+------------+--------+-------------------+----------+
| 2017 |          1 |      2 |            360.30 | USD      |
| 2017 |          2 |      2 |            263.90 | USD      |
| 2017 |          3 |      2 |            179.60 | USD      |
| 2017 |          4 |      2 |             99.60 | USD      |
| 2017 |          5 |      2 |            135.40 | USD      |
| 2017 |          6 |      2 |             10.10 | USD      |
| 2017 |          7 |      2 |             92.60 | USD      |
| 2019 |          1 |      1 |         531600.00 | VND      |
| 2019 |          1 |      2 |            347.20 | USD      |
| 2019 |          2 |      1 |         891400.00 | VND      |
| 2019 |          2 |      2 |            251.90 | USD      |
| 2019 |          3 |      1 |         824600.00 | VND      |
| 2019 |          3 |      2 |            184.60 | USD      |
| 2019 |          4 |      1 |         278200.00 | VND      |
| 2019 |          4 |      2 |            103.20 | USD      |
| 2019 |          5 |      1 |         569200.00 | VND      |
| 2019 |          5 |      2 |            142.70 | USD      |
| 2019 |          6 |      1 |          20800.00 | VND      |
| 2019 |          6 |      2 |             13.10 | USD      |
NORMAL  feature/part4  zsh [-]                    91%  599:27
```

For instance, knowing that the years across the 5 datasets range from 1995 to 2025, the INSERT INTO statement for the Year table manually inserts all years within that range. This logic was also applied to other tables with small numbers of records, such as TouristType, Region, TransportMode, and ExpenditureCategory.Relationship entities such as ArrivalByNationality, ArrivalByTransport, etc. required separate CSV files due to the large volume of data to be loaded. These CSV files were generated using Python scripts that format the original datasets into a format that fits the tables.

Design and implementation of queries:

Below are the 8 queries that were designed and implemented in Part3.sql:

1. What is the sum of expenditures per tourist across the years?

```
SELECT year,
  CASE
    WHEN currency = 'USD' THEN ROUND(SUM(expenditureAmount), 2)
    WHEN currency = 'VND' THEN ROUND(SUM(expenditureAmount) * 0.000039, 2)
    ELSE NULL
```

```
  END AS sum_expenditure_USD,
  CASE
    WHEN currency = 'USD' THEN 'International'
    WHEN currency = 'VND' THEN 'Domestic'
    ELSE NULL
  END AS tourist_type
FROM ExpenditureByCategory
GROUP BY year, currency;
```

This query would retrieve the amount of money international and domestic tourists spent in the tourism industry across the years. This query would be useful in comparing how much domestic and international tourists spend on tourism in Vietnam by providing an estimate of the total expenditure per tourist. The key assumption was that 1,000 VND = 0.039 USD. Below is a sample output, using the conversion rate from Google Finance as of 25/10/2026.

year	sum_expenditure_USD	tourist_type
2017	1141.50	International
2019	125.83	Domestic
2019	1151.70	International
2020	128.13	Domestic
2023	101.46	Domestic
2023	1449.70	International
2024	93.44	Domestic
2024	1164.90	International

2. Which provinces had the highest turnovers from tourism in 2024?

```
SELECT PT.year, P.name, PT.turnoverAmount
FROM ProvincialTurnover PT
INNER JOIN Province P
ON PT.provinceId = P.provinceId
WHERE PT.year = 2024
ORDER BY turnoverAmount DESC;
```

This query retrieves all provinces and orders them by turnover amount in descending order. This would tell us which provinces generated the highest turnover. The unit of turnoverAmount is in billion VND.

year	name	turnoverAmount
2024	Ha Noi	27800.70
2024	Ho Chi Minh city	27361.00
2024	Da Nang	5702.90
2024	Khanh Hoa	4610.50

2024	Quang Ninh	1187.10
2024	Kien Giang	1006.10
2024	Hai Phong	795.20
2024	Can Tho	724.50

3. Which regions had the highest turnovers from tourism in 2024?

```
SELECT PT.year, R.name, SUM(PT.turnoverAmount) AS total_turnover
FROM ProvincialTurnover PT
INNER JOIN Province P
ON PT.provinceId = P.provinceId
INNER JOIN Region R
ON P.regionId = R.regionId
WHERE PT.year = 2024
GROUP BY R.name
ORDER BY total_turnover DESC;
```

Similar to the last query, this query would return all regions ordered by turnover amount in descending order. This would allow us to compare the turnover amounts of regions.

year	name	total_turnover
2024	Red River Delta	31625.40
2024	South East	28434.50
2024	North Central and Central Coastal Areas	12762.70
2024	Mekong River Delta	2953.80
2024	Northern Midlands and Mountain Areas	1045.50
2024	Central Highlands	168.40

4. How many Australian tourists visited Vietnam each year from 2015 onwards?

```
SELECT A.year, A.arrivalCount
FROM ArrivalByNationality A
INNER JOIN Country C
ON A.countryId = C.countryId
WHERE C.name = 'Australian'
AND A.year >= 2015;
```

This query returns the arrival count of a given nationality from a specific year onwards. This provides the user with a clear and concise view of changes in arrival counts over the years.

year	arrivalCount

2015	303700
2016	320700
2017	370400
2018	386900
2019	383500
2020	92800
2021	1000
2022	144700
2023	390100
2024	490900
2025	548500

5. For international tourists, which expenditure categories exceed the average expenditure across all categories in the same year?

```
SELECT E.year, EC.category, E.expenditureAmount
FROM ExpenditureByCategory E
INNER JOIN ExpenditureCategory EC
ON E.categoryId = EC.categoryId
INNER JOIN TouristType T
ON E.typeId = T.typeId
WHERE T.type = 'International'
AND E.expenditureAmount > (
  SELECT AVG(E1.expenditureAmount)
  FROM ExpenditureByCategory E1
  WHERE E.year = E1.year
  AND E.typeId = E1.typeId
)
ORDER BY E.year;
```

By using a subquery to calculate the average expenditure across all categories in the same year and tourist type, we can identify which categories have amounts above the average. This tells the user which categories, on average, yield more money.

year	category	expenditureAmount
2017	Accommodation	360.30
2017	Food	263.90
2017	Transport	179.60
2019	Accommodation	347.20
2019	Food	251.90

6. How did international arrivals by air, road, and water change between 2019 and 2024?

```
SELECT T.mode, A.arrivalCount, A2.arrivalCount,
(A2.arrivalCount / A.arrivalCount) * 100 - 100 AS growth_pct
FROM ArrivalByTransport A
INNER JOIN ArrivalByTransport A2
ON A.transportId = A2.transportId
INNER JOIN TransportMode T
ON A.transportId = T.transportId
WHERE A.year = '2019'
AND A2.year = '2024';
```

A self-join has been used here to match the same transport mode across two years. This allows the differences between subsequent years to be calculated, enabling the calculation of growth percentage.

mode	arrivalCount_2019	arrivalCount_2024	growth_pct
Air way	14377500	14844100	3.2500
Water way	264100	248100	-6.0600
Road	3367000	2491700	-26.0000

7. Which countries had more arrivals in Vietnam than the average across all nationalities in 2024?

```
SELECT A.year, C.name, A.arrivalCount
FROM ArrivalByNationality A
INNER JOIN Country C
ON A.countryId = C.countryId
WHERE A.year = 2024
AND A.arrivalCount > (
  SELECT AVG(arrivalCount)
  FROM ArrivalByNationality
  WHERE year = A.year
);
```

The subquery is used to calculate the average tourist arrival count across the selected year, enabling us to determine which countries had higher-than-average arrival counts.

year	name	arrivalCount
2024	Chinese	3738100
2024	Taiwanese	1288900
2024	Korean	4568900
2024	American	779800

8. Which provinces have names containing 'Quang', and what were their tourism turnovers in 2023?

```
SELECT PT.year, P.name, PT.turnoverAmount
FROM ProvincialTurnover PT
INNER JOIN Province P
ON PT.provinceId = P.provinceId
WHERE P.name LIKE "%Quang%"
AND PT.year = 2023;
```

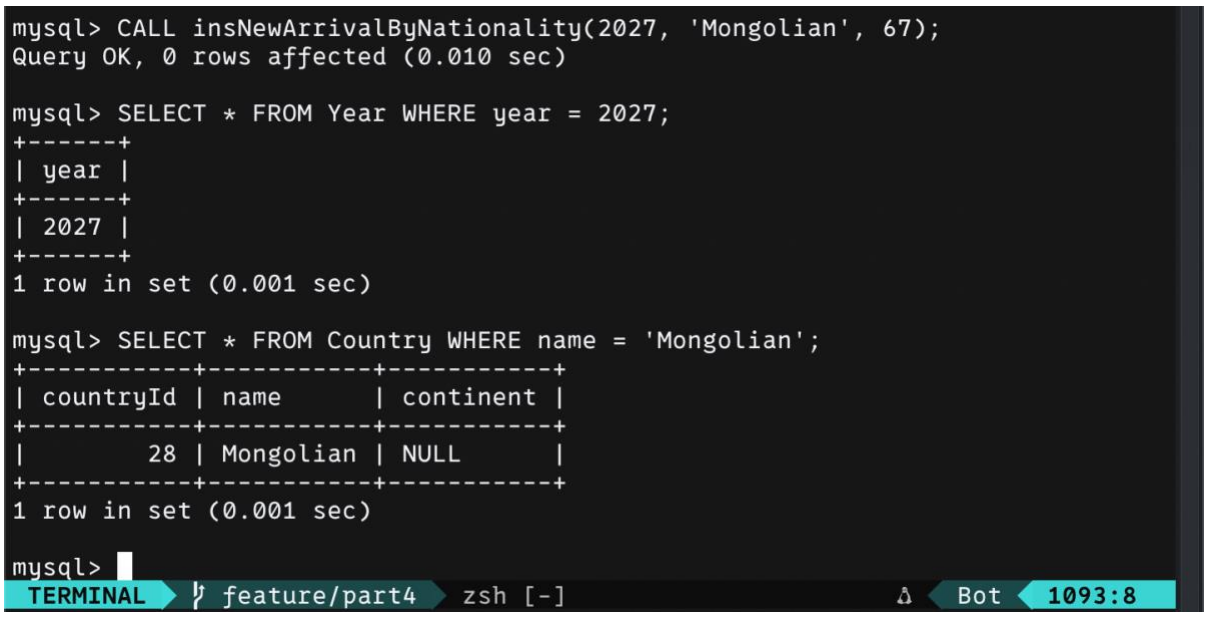
This query allows us to match a substring across all provinces within the database. Having the % on both sides matches the name everywhere, e.g. 'Quang Ninh' and 'Tuyen Quang'.

year	name	turnoverAmount
2023	Quang Ninh	1046.00
2023	Tuyen Quang	3.50
2023	Quang Binh	318.30
2023	Quang Tri	28.20

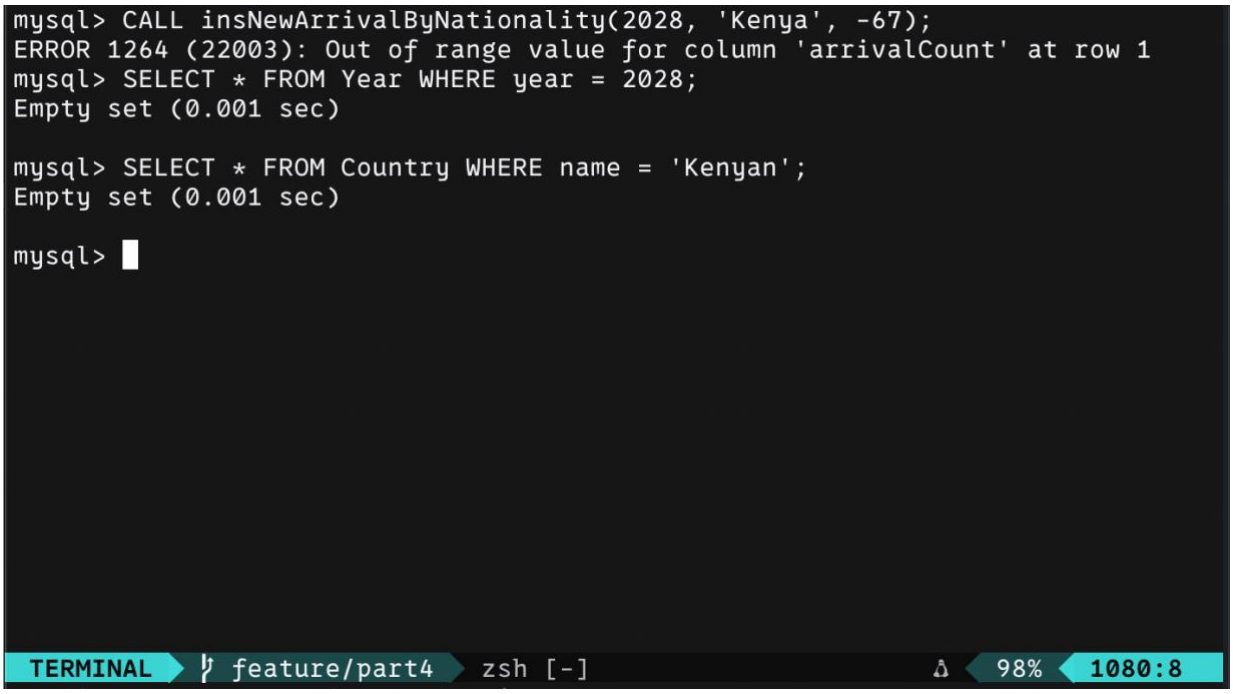
Advanced concepts:

Procedures: Insert a new record into ArrivalByNationality, and have the procedure check whether the country string exists in Country. If not, insert it. Then do the same for the year before inserting the new record with the data.

- An example of a valid call for the procedure using a new 'year' entry and a new 'country' entry. Both new entries can be seen inserted into Year and Country, respectively:

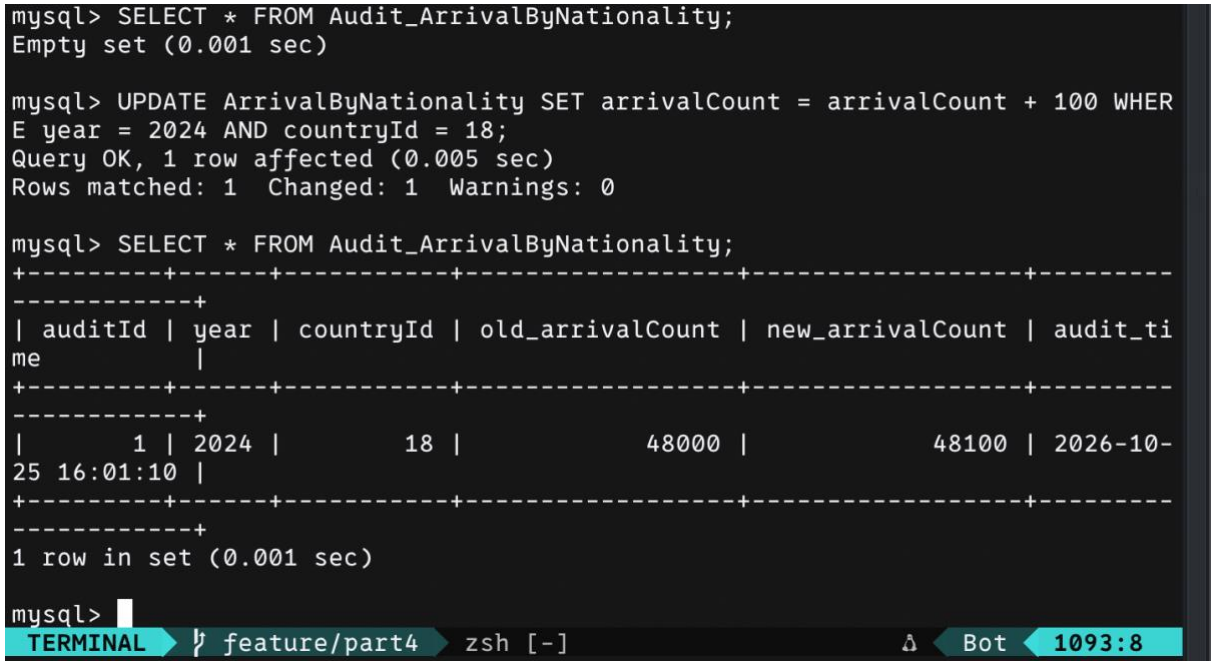


- An example of an invalid call, where while both ‘year’ and ‘country’ are valid and can be inserted, the arrivalCount is invalid, causing the procedure to roll back:

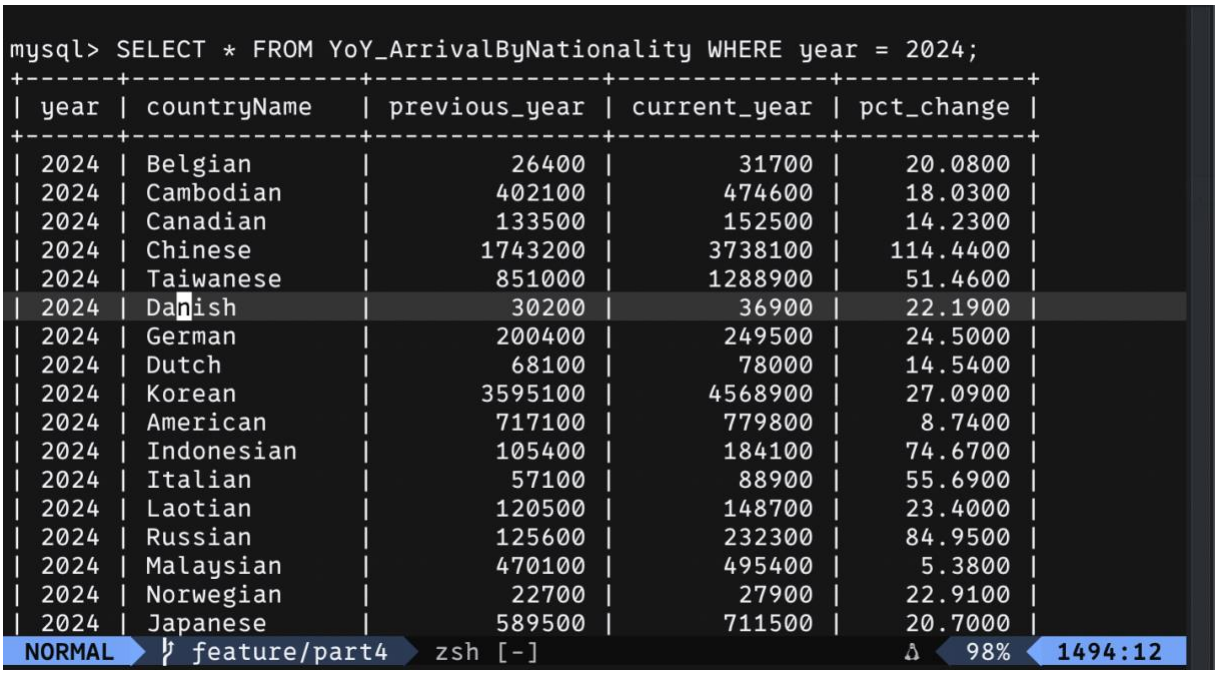


Triggers: An AFTER UPDATE trigger that inserts information about updates to ArrivalByNationality, tracking the old values, new values, and the timestamps.

- An example of the Audit_ArrivalByNationality table after one of the records in ArrivalByNationality is updated:



Views: A view that shows the YoY % Change between the records within ArrivalByNationality. Only show the change for records with an existing record for the previous year.

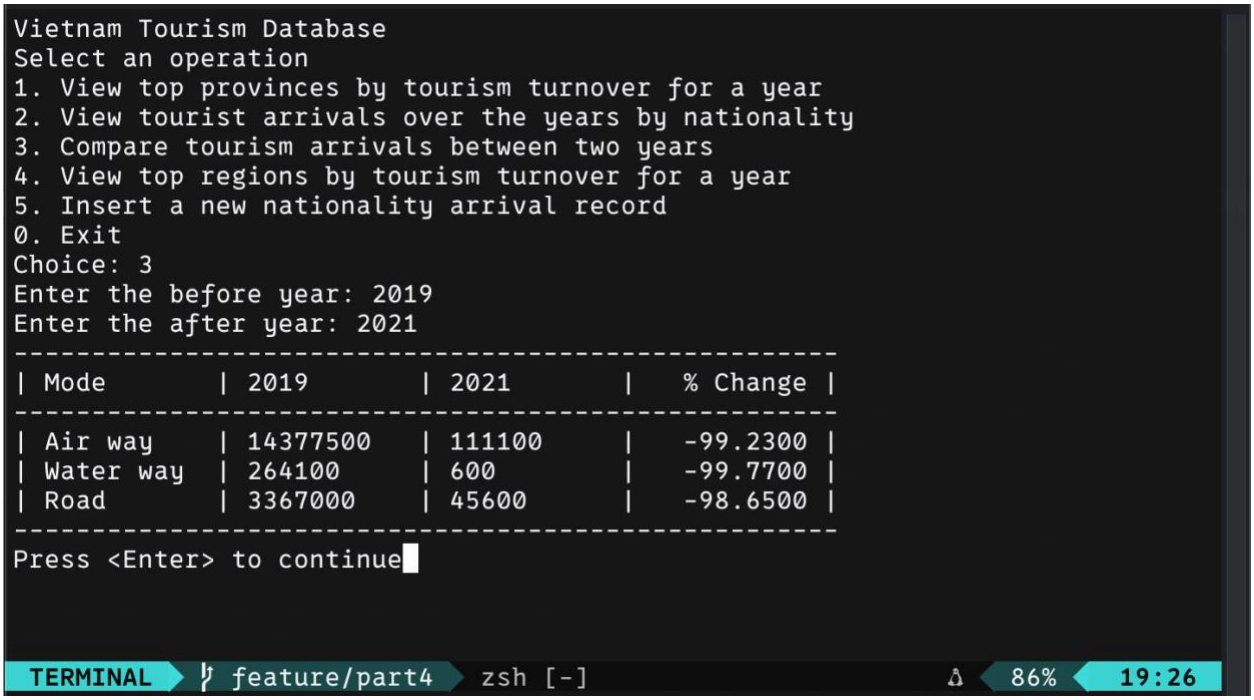


```
mysql> SELECT * FROM YoY_ArrivalByNationality WHERE year = 2024;
+------+---------------+---------------+--------------+------------+
| year | countryName   | previous_year | current_year | pct_change |
+------+---------------+---------------+--------------+------------+
| 2024 | Belgian       |         26400 |        31700 |    20.0800 |
| 2024 | Cambodian     |        402100 |       474600 |    18.0300 |
| 2024 | Canadian      |        133500 |       152500 |    14.2300 |
| 2024 | Chinese       |       1743200 |      3738100 |   114.4400 |
| 2024 | Taiwanese     |        851000 |      1288900 |    51.4600 |
| 2024 | Danish        |         30200 |        36900 |    22.1900 |
| 2024 | German        |        200400 |       249500 |    24.5000 |
| 2024 | Dutch         |         68100 |        78000 |    14.5400 |
| 2024 | Korean        |       3595100 |      4568900 |    27.0900 |
| 2024 | American      |        717100 |       779800 |     8.7400 |
| 2024 | Indonesian    |        105400 |       184100 |    74.6700 |
| 2024 | Italian       |         57100 |        88900 |    55.6900 |
| 2024 | Laotian       |        120500 |       148700 |    23.4000 |
| 2024 | Russian       |        125600 |       232300 |    84.9500 |
| 2024 | Malaysian     |        470100 |       495400 |     5.3800 |
| 2024 | Norwegian     |         22700 |        27900 |    22.9100 |
| 2024 | Japanese      |        589500 |       711500 |    20.7000 |
NORMAL  feature/part4  zsh [-]                          98%  1494:12
```

Database connection with Python:

The database was connected to a Python program through the mysql-connector-python library. The program allows the user to enter the username and password to access their database on localhost. The program then presents the user with 5 different queries which they can execute on the database. Some of these queries also allow the user to enter custom values to query different results.

- Below is a screenshot of the menu screen of the Python program after the user has successfully logged in. It also shows the user choosing option number 3, entering the required data, and the output table being displayed.



Discussion and reflection:

The concept of transactions is used by the procedure 'insNewArrivalByNationality', where the three separate insertions are either all valid or none go through, and the tables are rolled back to the previous state. For instance, the user calls 'insNewArrivalByNationality', providing a valid year and country but an invalid arrival count. Without transaction handling, after inserting the year and country into their corresponding tables and then failing at the final insertion, the inserted year and country would have remained. With transaction handling, the failure of the final insertion will cause both the Year and Country tables to roll back to their previous state. Therefore, either all three insertions are valid, or none will take place.

In this assignment, I have implemented a database system that incorporates data from Vietnam's National Statistics Office (NSO), specifically targeting tourism-related data. I have designed a database system by creating ER diagrams and database schemas, and then implemented them using SQL. The data retrieved from the NSO are then transformed, restructured and re-organised to fit the implemented table structures before being loaded into the tables. I have also created a Python program that connects to a MySQL database, allowing users to run predefined queries to access and view data.

While various challenges were encountered throughout this assignment, the main one was the preprocessing of the data. This was mainly caused by the data having different formats from the designed and implemented tables. This challenge was overcome using Python scripts that read the data and rearranged it into the desired format. The limitations of my work include the limited error handling in the Python program, where not all possible error cases are accounted for. Similarly, the procedure and trigger were not implemented to raise any errors, which and cause the program to crash when unexpected data is entered. If more time were allocated, this work could be improved by adding more edge-case handling and creating more useful derived attributes.